Typst changes from origin/main (d9a59c1f) to a1eb4672

Typst changes from origin/main (d9a59c1f) to a1eb4672 1
thesis/content/01-introduction.typ 1
thesis/content/02-background.typ 2
thesis/content/03-program-model.typ 3
thesis/content/04-traces.typ 4
thesis/content/05-domains.typ 5
thesis/content/07-cooperation.typ 6
thesis/content/08-equations.typ 7
thesis/content/09-solving.typ 7
thesis/content/11-instances.typ 8
thesis/content/12-executable.typ 9
thesis/content/13-evaluation.typ 10
thesis/content/15-conclusion.typ 10
thesis/content/abstract.typ 11
thesis/content/ai-use.typ 11
thesis/content/appendix-tooling.typ 13
thesis/content/reading-guide.typ 15
thesis/lib/tum.typ 15

thesis/content/01-introduction.typ

d9a59c1f → a1eb4672

593 old lines, 593 new lines

99% similar lines **+4** **-4** **2 changed blocks**

Old	d9a59c1f	New	a1eb4672
1	`#import "../lib/code.typ": (`	1	`#import "../lib/code.typ": (`
2	`  fixture, isaconst, isalocale, isathm, isatype, listing, oblig, rebase, site,`	2	`  fixture, isaconst, isalocale, isathm, isatype, listing, oblig, rebase, site,`
3	`)`	3	`)`
	54 unchanged lines hidden		
58	`control-flow graph to the solver's result, is connected to source executions by`	58	`control-flow graph to the solver's result, is connected to source executions by`
59	`proof. Of the semantics, only #isaconst("pstep") therefore needs an adequacy`	59	`proof. Of the semantics, only #isaconst("pstep") therefore needs an adequacy`
60	`argument instead of a proof. That argument states which fragment of C it models, where it departs`	60	`argument instead of a proof. That argument states which fragment of C it models, where it departs`
61	`from C11 (@sec:vimp-vs-c), and why no existing verified semantics serves as the`	61	`from C11 (@sec:vimp-vs-c), and why no existing verified semantics can be used`
62	`anchor.`	62	`instead.`
63		63	
64	`== Sound static analysis and its verdicts <sec:intro-static>`	64	`== Sound static analysis and its verdicts <sec:intro-static>`
65		65	
	156 unchanged lines hidden		
222	`happen, for instance, that \`eval\` does nothing, so its result can miss`	222	`happen, for instance, that \`eval\` does nothing, so its result can miss`
223	`executions. The authors call such analyses _soundy_. They treat this as an`	223	`executions. The authors call such analyses _soundy_. They treat this as an`
224	`engineering choice for precision and scalability, and they ask the authors of`	224	`engineering choice for precision and scalability, and they ask the authors of`
225	`such analyses to clearly`	225	`such analyses to state explicitly`
226	`state the nature and extent of the unsoundness @livshits15[p. 45]. Goblint,`	226	`the nature and extent of the unsoundness @livshits15[p. 45]. Goblint,`
227	`for instance, documents such assumptions. It assumes that pointer arithmetic`	227	`for instance, documents such assumptions. It assumes that pointer arithmetic`
228	`does not overflow and, by default, that array reads stay in bounds, and it notes`	228	`does not overflow and, by default, that array reads stay in bounds, and it notes`
229	`that this list is likely incomplete @goblint-assumptions.`	229	`that this list is likely incomplete @goblint-assumptions.`

	361 unchanged lines hidden		
591	concludes (@ch:conclusion). @sec:isabelle explains the Isabelle syntax of the	591	concludes (@ch:conclusion). @sec:isabelle explains the Isabelle syntax of the
592	listings, @app:notation lists the notation, and @sec:voblint-repo describes the repository, its	592	listings, @app:notation lists the notation, and @sec:voblint-repo describes the repository, its
593	checks, and the source tooling built for this work.	593	checks, and the source tooling built for this work.

thesis/content/02-background.typ

d9a59c1f → a1eb4672

759 old lines, 759 new lines

99% similar lines **+9** **-9** **6 changed blocks**

Old	d9a59c1f	New	a1eb4672
1	#import "../lib/math.typ": *	1	#import "../lib/math.typ": *
2	#import "../lib/code.typ": (	2	#import "../lib/code.typ": (
3	isacmd, isaconst, isai, isalocale, isasession, isathm, isatype, listing, playground-link,	3	isacmd, isaconst, isai, isalocale, isasession, isathm, isatype, listing, playground-link,
	131 unchanged lines hidden		
135	that the step loses nothing. This chapter introduces the ideas needed to state	135	that the step loses nothing. This chapter introduces the ideas needed to state
136	these steps. It first describes what a program can actually do, then how an	136	these steps. It first describes what a program can actually do, then how an
137	analysis describes the same behavior more coarsely, and finally how a solver	137	analysis describes the same behavior more coarsely, and finally how a solver
138	computes this description. One small counting loop serves as the example	138	computes this description. A loop that counts from 0 to 5 is the running
139	throughout.	139	example for all three.
140		140	
141	== Control-flow graphs and the collecting semantics <sec:collecting>	141	== Control-flow graphs and the collecting semantics <sec:collecting>
142		142	
	287 unchanged lines hidden		
430	whose post-fixpoints appear in @fig:lattice-fixpoints. A solution assigns an	430	whose post-fixpoints appear in @fig:lattice-fixpoints. A solution assigns an
431	interval to every unknown such that all four inequalities hold, and	431	interval to every unknown such that all four inequalities hold, and
432	@fig:loop-solutions shows two. Both contain the collecting semantics	432	@fig:loop-solutions shows two. Both contain the collecting semantics
433	(${0, dots, 5}$ at h), and the second is less precise at h and e, so a	433	(${0, dots, 5}$ at h), and the second is less precise at h and e, so not
434	solution need not be the least one.	434	every solution is the least one.
435		435	
436	In general, let Unk be a set of unknowns and $sol : Unk -> A$ a	436	In general, let Unk be a set of unknowns and $sol : Unk -> A$ a
437	_valuation_, which assigns a domain element to every unknown. Valuations are	437	_valuation_, which assigns a domain element to every unknown. Valuations are
	67 unchanged lines hidden		
505	two @cousot77[§9.3.4] @cousot92plilp[§4]:	505	two @cousot77[§9.3.4] @cousot92plilp[§4]:
506	$ b lle a narrow b lle a. $	506	$ b lle a narrow b lle a. $
507	In the counting loop, the interval narrowing reaches $absval(ivl(0, 5))$, but in general,	507	In the counting loop, the interval narrowing reaches $absval(ivl(0, 5))$, but in general,
508	narrowing need not recover the least fixpoint. Both steps appear in	508	narrowing does not always recover the least fixpoint. Both steps appear in
509	@fig:widening-phases. The verified TD solver applies both operators in a single	509	@fig:widening-phases. The verified TD solver applies both operators in a single
510	update, _warrowing_ (#isaconst("warrow")), shown in @fig:widening-warrow. It	510	update, _warrowing_ (#isaconst("warrow")), shown in @fig:widening-warrow. It
511	narrows when the candidate lies below the current value and widens otherwise	511	narrows when the candidate lies below the current value and widens otherwise
	28 unchanged lines hidden		
540	warrowing update lies above the candidate	540	warrowing update lies above the candidate
541	(#isathm("warrowing_properties", thy: "Warrowing")), so an unknown whose value	541	(#isathm("warrowing_properties", thy: "Warrowing")), so an unknown whose value
542	no longer changes satisfies its inequality. The solver's correctness theorem	542	no longer changes satisfies its inequality. The solver's correctness theorem

543	uses only this and holds whether or not the right-hand sides are monotone	543	uses only this and holds whether the right-hand sides are monotone
544	(@sec:td). Monotonicity is needed for the classical theory of least fixpoints,	544	(@sec:td). Monotonicity is needed for the classical theory of least fixpoints,
545	but not to certify that a returned valuation is a post-solution. Termination	545	but not to certify that a returned valuation is a post-solution. Termination
546	needs more. A classical widening is chosen so that repeated widening stabilizes	546	needs more. A classical widening is chosen so that repeated widening stabilizes
	82 unchanged lines hidden		
629	only _global unknowns_ Unk_G, which have no right-hand side and receive only	629	only _global unknowns_ Unk_G, which have no right-hand side and receive only
630	contributions. From here on, Unk denotes the _local_ unknowns, which keep	630	contributions. From here on, Unk denotes the _local_ unknowns, which keep
631	their right-hand sides, and a valuation sol assigns values to both kinds. A	631	their right-hand sides, and a valuation sol assigns values to both kinds. A
632	global unknown need not correspond to a global variable of the program. It can	632	global unknown does not have to correspond to a global variable of the program. It can
633	hold any fact to which several right-hand sides contribute. In Voblint, for	633	hold any fact to which several right-hand sides contribute. In Voblint, for
634	instance, the entry state of a callee in one context is a global unknown, and	634	instance, the entry state of a callee in one context is a global unknown, and
635	every call that enters this context publishes its entry state to it	635	every call that enters this context publishes its entry state to it
	37 unchanged lines hidden		
673	unknown that it read has changed since then. When the value of y changes	673	unknown that it read has changed since then. When the value of y changes
674	later, every unknown that read it is _destabilized_ and evaluated again. The	674	later, every unknown that read it is _destabilized_ and evaluated again. The
675	unknowns at which a read closes a cycle are the solver's _widening points_	675	unknowns at which a read closes a cycle are the solver's _widening points_
676	@seidl21. TD detects them while it solves @apinis16[§4], so they need not	676	@seidl21. TD detects them while it solves @apinis16[§4], so they can
677	coincide with the loop heads of the control-flow graph. At a widening point, it	677	differ from the loop heads of the control-flow graph. At a widening point, it
678	updates the old value with the new one by warrowing. The	678	updates the old value with the new one by warrowing. The
679	#link(playground-link(_counting-loop, trace: "verbose"))[playground] replays this run on the counting loop step by step. Contributions published to a	679	#link(playground-link(_counting-loop, trace: "verbose"))[playground] replays this run on the counting loop step by step. Contributions published to a
680	global unknown are merged into its value by an _update rule_	680	global unknown are merged into its value by an _update rule_
	76 unchanged lines hidden		
757	and V is called the _solved set_. Unlike some TD variants without side effects	757	and V is called the _solved set_. Unlike some TD variants without side effects
758	@tilscher26jar, this solver has no termination proof (@sec:isabelle explains	758	@tilscher26jar, this solver has no termination proof (@sec:isabelle explains
759	how Isabelle defines such a function).	759	how Isabelle defines such a function).

thesis/content/03-program-model.typ

d9a59c1f → a1eb4672

894 old lines, 894 new lines

99% similar lines +5 -5 **4 changed blocks**

Old	d9a59c1f	New	a1eb4672
1	#import "@preview/fletcher:0.5.8" as fletcher: diagram, edge, node	1	#import "@preview/fletcher:0.5.8" as fletcher: diagram, edge, node
2	#import "@preview/curryst:0.6.0": prooftree, rule	2	#import "@preview/curryst:0.6.0": prooftree, rule
3	#import "@preview/cetz:0.5.2"	3	#import "@preview/cetz:0.5.2"
	23 unchanged lines hidden		
27	the theorem, and a verdict would say nothing about the source program. This	27	the theorem, and a verdict would say nothing about the source program. This
28	chapter closes the gap with the first link of the soundness chain. Every	28	chapter closes the gap with the first link of the soundness chain. Every
29	source execution of a well-formed program is matched by a graph execution that holds the	29	source execution of a well-formed program is matched by a graph execution that holds the
30	same store (@sec:csim). The program of @fig:program-to-equations, in which	30	same store (@sec:csim). The running example is the program of
31	`main` calls `bump(5)` and `bump(4)`, serves as the running example.	31	@fig:program-to-equations, in which `main` calls `bump(5)` and `bump(4)`.
32		32	
33	// Node names and edge labels come from registered `--dot` output	33	// Node names and edge labels come from registered `--dot` output
34	// (lib/cfg-graphs.typ).	34	// (lib/cfg-graphs.typ).

	383 unchanged lines hidden		
418		418	
419	The root activation is modeled without a #ctor("Restore"), since `main` has no	419	The root activation is modeled without a #ctor("Restore"), since `main` has no
420	caller to return to, and a bare #ctor("Unwind") has no step	420	caller to return to, and a bare #ctor("Unwind") has no step
421	(#isathm("pstep_Unwind_stuck")). This is why the source contract rejects	421	(#isathm("pstep_Unwind_stuck")). The source contract therefore rejects
422	#keyw("return") in `main` (#isaconst("no_return")). A source execution starts from	422	#keyw("return") in `main` (#isaconst("no_return")). A source execution starts from
423	the configuration $(#isaconst("main_body") thin Pi, s_0, [])$, with an initial	423	the configuration $(#isaconst("main_body") thin Pi, s_0, [])$, with an initial
424	store s_0 in #isaconst("cinit_stores") (@sec:vimp-vs-c).	424	store s_0 in #isaconst("cinit_stores") (@sec:vimp-vs-c).
	230 unchanged lines hidden		
655		655	
656	The compiler produces a static object. To say that it preserves the behavior	656	The compiler produces a static object. To say that it preserves the behavior
657	of a program, the graph needs an execution of its own, which @sec:csim then	657	of a program, the graph needs an execution of its own, which @sec:csim then
658	compares with #isaconst("pstep"). Within one procedure, an execution simply follows	658	compares with #isaconst("pstep"). Within one procedure, an execution follows
659	edges, so a node and a store would describe its state. Calls break this	659	edges, so a node and a store would describe its state. Calls break this
660	picture. A call edge leads to the callee's entry, but the callee ends at its	660	picture. A call edge leads to the callee's entry, but the callee ends at its
661	result node, which all callers share and which no edge leaves (@sec:cfg). The	661	result node, which all callers share and which no edge leaves (@sec:cfg). The
	126 unchanged lines hidden		
788	`x = 1` is related to both branch nodes. The node must come from	788	`x = 1` is related to both branch nodes. The node must come from
789	the run.	789	the run.
790		790	
791	This is why every command carries an annotation (@sec:vimp). A parsed program	791	Every command therefore carries an annotation (@sec:vimp). A parsed program
792	leaves it empty, and the proof fills it in. The program's _elaboration_	792	leaves it empty, and the proof fills it in. The program's _elaboration_
793	(#isaconst("elab_pt")) annotates each statement with the nodes the compiler	793	(#isaconst("elab_pt")) annotates each statement with the nodes the compiler
794	emits for it. The elaboration compiles to the same graph	794	emits for it. The elaboration compiles to the same graph
	97 unchanged lines hidden		
892	calls with results. Since VIMP also has the shape of Goblint's normalized	892	calls with results. Since VIMP also has the shape of Goblint's normalized
893	input, a C front end with a verified translation could reuse the framework,	893	input, a C front end with a verified translation could reuse the framework,
894	although each omitted feature would need its own preservation argument.	894	although each omitted feature would need its own preservation argument.

thesis/content/04-traces.typ

d9a59c1f → a1eb4672

614 old lines, 613 new lines

99% similar lines **+6** **-7** **3 changed blocks**

Old	d9a59c1f	New	a1eb4672
1	#import "@preview/fletcher:0.5.8" as fletcher: diagram, edge, node	1	#import "@preview/fletcher:0.5.8" as fletcher: diagram, edge, node
2	#import "../lib/math.typ": *	2	#import "../lib/math.typ": *
3	#import "../lib/theme.typ": vb	3	#import "../lib/theme.typ": vb
	234 unchanged lines hidden		
238	a lock may incorporate a trace of whichever thread last released the mutex. An	238	a lock may incorporate a trace of whichever thread last released the mutex. An
239	activation trace holds the path of a single activation and refers to at most	239	activation trace holds the path of a single activation and refers to at most
240	two other traces, and a $ctor("Resume")$ accepts only the callee that this	240	two other traces, and a $ctor("Resume")$ accepts only the callee that this
241	very activation created. Activations do not run concurrently, so nothing	241	activation itself created. Activations do not run concurrently, so nothing
242	corresponds to reading globals written by another thread.	242	corresponds to reading globals written by another thread.
243		243	

244	=== Valid traces follow graph steps #thy-badge("Voblint_CFG", "Activation_Trace_Def") <sec:valid>	244	=== Valid traces follow graph steps #thy-badge("Voblint_CFG", "Activation_Trace_Def") <sec:valid>
	186 unchanged lines hidden		
431	@sharir81 @seidl12compiler[§§2.6, 2.9]. Entry-state contexts follow Goblint's	431	@sharir81 @seidl12compiler[§§2.6, 2.9]. Entry-state contexts follow Goblint's
432	partial tabulation, which tabulates procedure summaries only for the calling	432	partial tabulation, which tabulates procedure summaries only for the calling
433	contexts the analysis meets @saan26phd[§3.3] @schwarz25phd[§5, p. 110]. Unlike	433	contexts the analysis meets @saan26phd[§3.3] @schwarz25phd[§5, p. 110]. Unlike
434	bounded call strings, they need not form a finite set, so the analysis may not	434	bounded call strings, they can form an infinite set, so the analysis may not
435	terminate (@sec:certificate). The traces themselves store no context, since	435	terminate (@sec:certificate). The traces themselves store no context, since
436	the concrete semantics should not change with the analysis. The policy is	436	the concrete semantics should not change with the analysis. The policy is
437	instead a parameter of the theorems, which therefore hold for every policy. It	437	instead a parameter of the theorems, which therefore hold for every policy. It
	116 unchanged lines hidden		
554	#oblig("RETURN") reads.],	554	#oblig("RETURN") reads.],
555	) <fig:contract>	555	) <fig:contract>
556		556	
557	#oblig("TOTAL") closes a gap that the other four obligations leave.	557	#oblig("CALL") and #oblig("RETURN") constrain only the contexts that the policy
558	#oblig("CALL") and #oblig("RETURN") only speak about the contexts that the	558	admits. If a call from a store in the cover gets no context at all, both
559	policy admits. If a call from a store in the cover gets no context at all, both	559	hold vacuously, and no obligation requires the cover at the call's
560	hold vacuously, and nothing puts the store after the return into the cover at	560	continuation to contain the store the call returns with. The resumed caller, however, still carries its contexts there
561	the continuation. The resumed caller, however, still carries its contexts there		
562	(#isathm("resume_keeps_context")), so the cover can miss a store that the	561	(#isathm("resume_keeps_context")), so the cover can miss a store that the
563	program reaches (#isathm("total_dropped_unsound")). #oblig("TOTAL") therefore requires the	562	program reaches (#isathm("total_dropped_unsound")). #oblig("TOTAL") therefore requires the
564	policy to admit at least one callee context for every call from a store	563	policy to admit at least one callee context for every call from a store
	47 unchanged lines hidden		
612	preserves. The following chapters build such analyses from abstract domains	611	preserves. The following chapters build such analyses from abstract domains
613	(@ch:domains), transfer functions (@ch:analysis-interface), and a solved	612	(@ch:domains), transfer functions (@ch:analysis-interface), and a solved
614	equation system (@ch:equations, @ch:solving).	613	equation system (@ch:equations, @ch:solving).

thesis/content/05-domains.typ

d9a59c1f → a1eb4672

1003 old lines, 1003 new lines

100% similar lines **+3** **-3** **3 changed blocks**

Old	d9a59c1f	New	a1eb4672
1	#import "@preview/fletcher:0.5.8" as fletcher: diagram, edge, node	1	#import "@preview/fletcher:0.5.8" as fletcher: diagram, edge, node
2	#import "../lib/math.typ": *	2	#import "../lib/math.typ": *
3	#import "../lib/code.typ": (	3	#import "../lib/code.typ": (
	518 unchanged lines hidden		
522	$y <= 5$, but not $x <= y$, since it also admits $x = 5, y = 0$. A relational	522	$y <= 5$, but not $x <= y$, since it also admits $x = 5, y = 0$. A relational
523	description that records $x <= y$ shows exactly that fact, but it bounds	523	description that records $x <= y$ shows exactly that fact, but it bounds
524	neither variable, so it also admits $x = y = -7$. Therefore, neither description is stronger than the other. Together, however, they describe exactly	524	neither variable, so it also admits $x = y = -7$. Therefore, neither description is stronger than the other. Together, however, they describe exactly
525	the given set.	525	the stores with $0 <= x <= y <= 5$.
526		526	
527	#figure(	527	#figure(
528	cetz.canvas(length: 0.5cm, {	528	cetz.canvas(length: 0.5cm, {

	39 unchanged lines hidden		
568	a whole state to a set of stores instead of each variable to a set of	568	a whole state to a set of stores instead of each variable to a set of
569	integers. The interface of this chapter admits such a state, because the	569	integers. The interface of this chapter admits such a state, because the
570	solver and the framework ask only for order, join, and a monotone	570	solver and the framework ask only for order, join, and a monotone
571	concretization, never for a map from variables to abstract values. The	571	concretization. Neither requires the state to map variables to abstract values. The
572	relational state of the order analysis instantiates the solver's classes and	572	relational state of the order analysis instantiates the solver's classes and
573	#isalocale("executable_domain") without being a numeric domain. The	573	#isalocale("executable_domain") without being a numeric domain. The
574	remaining sections of this chapter develop the non-relational case, and	574	remaining sections of this chapter develop the non-relational case, and
	70 unchanged lines hidden		
645	every empty state as unreachable is correct at the moment of the test, but the	645	every empty state as unreachable is correct at the moment of the test, but the
646	information does not survive the next step. Joining two differently empty	646	information does not survive the next step. Joining two differently empty
647	states can give a nonempty one (@fig:domain-reachability). This is sound,	647	states can give a nonempty one (@fig:domain-reachability). This is sound,
648	since it only adds stores, but it loses the fact that the point is unreachable	648	since it only adds stores, but it loses the information that the point is unreachable
649	and is thus imprecise.	649	and is thus imprecise.
650		650	
651	#let _r = claim-row("dom-disjunct-sign", "13:5")	651	#let _r = claim-row("dom-disjunct-sign", "13:5")
	349 unchanged lines hidden		
1001	does, or by weakening the join to an arbitrary upper bound throughout the	1001	does, or by weakening the join to an arbitrary upper bound throughout the
1002	solver and the framework and reproving the results that rely on leastness.	1002	solver and the framework and reproving the results that rely on leastness.
1003	Both are left for future work.	1003	Both are left for future work.

thesis/content/07-cooperation.typ

d9a59c1f → a1eb4672

374 old lines, 374 new lines

100% similar lines **+1** **-1** **1 changed blocks**

Old	d9a59c1f	New	a1eb4672
1	#import "@preview/fletcher:0.5.8": diagram, edge, node	1	#import "@preview/fletcher:0.5.8": diagram, edge, node
2	#import "../lib/theme.typ": vb	2	#import "../lib/theme.typ": vb
3	#import "../lib/code.typ": isaconst, isai, isalocale, isathm, isatype, listing, oblig, thy-badge	3	#import "../lib/code.typ": isaconst, isai, isalocale, isathm, isatype, listing, oblig, thy-badge
	27 unchanged lines hidden		
31	}	31	}
32	```)	32	```)
33		33	
34	A reader easily sees that the check holds in every run, since both guards hold	34	The check holds in every run, since both guards hold
35	at the assignment, so $x = y$ and `z` is 1. No single analysis, however, can	35	at the assignment, so $x = y$ and `z` is 1. No single analysis, however, can
36	prove it. Interval keeps one range per variable. After the guards, both `x` and `y` are	36	prove it. Interval keeps one range per variable. After the guards, both `x` and `y` are
37	still unbounded, so `x == y` evaluates to $absval(ivl(0, 1))$ and the check stays	37	still unbounded, so `x == y` evaluates to $absval(ivl(0, 1))$ and the check stays
	334 unchanged lines hidden		
372	analysis requires only its own proof, and the proofs of the others stay	372	analysis requires only its own proof, and the proofs of the others stay
373	unchanged. @ch:equations discharges the obligations of @ch:traces for any	373	unchanged. @ch:equations discharges the obligations of @ch:traces for any
374	analysis that meets the analysis contract, the combination included.	374	analysis that meets the analysis contract, the combination included.

thesis/content/08-equations.typ

d9a59c1f → a1eb4672

519 old lines, 519 new lines

100% similar lines **+1** **-1** **1 changed blocks**

Old	d9a59c1f	New	a1eb4672
1	#import "@preview/fletcher:0.5.8": diagram, edge, node	1	#import "@preview/fletcher:0.5.8": diagram, edge, node
2	#import "../lib/theme.typ": vb	2	#import "../lib/theme.typ": vb
3	#import "../lib/code.typ": (	3	#import "../lib/code.typ": (
	79 unchanged lines hidden		
83	$ctxh(u, c, e)$ of the call site u, the caller context c, and the entry	83	$ctxh(u, c, e)$ of the call site u, the caller context c, and the entry
84	state e. Under entry-state contexts, the choice depends on e, which is	84	state e. Under entry-state contexts, the choice depends on e, which is
85	known only while the equations are solved. The set of calls that contribute to	85	known only while the equations are solved. The set of calls that contribute to
86	a particular callee context is therefore not known in advance, and the	86	a callee context is therefore not known in advance, and the
87	callee's entry cannot list them in its equation. Each call instead publishes	87	callee's entry cannot list them in its equation. Each call instead publishes
88	its entry state to the callee as a side effect (@sec:side-effects).	88	its entry state to the callee as a side effect (@sec:side-effects).
89		89	
	427 unchanged lines hidden		
517	@tab:eq-obligations each. Instantiated for the policies of @tab:eq-policies,	517	@tab:eq-obligations each. Instantiated for the policies of @tab:eq-policies,
518	the theorem covers the analyzer's equations, and @ch:solving computes such a	518	the theorem covers the analyzer's equations, and @ch:solving computes such a
519	post-solution.	519	post-solution.

thesis/content/09-solving.typ

d9a59c1f → a1eb4672

351 old lines, 352 new lines

96% similar lines **+13** **-12** **3 changed blocks**

Old	d9a59c1f	New	a1eb4672
1	#import "@preview/fletcher:0.5.8": diagram, edge, node	1	#import "@preview/fletcher:0.5.8": diagram, edge, node
2	#import "@preview/cetz:0.5.2"	2	#import "@preview/cetz:0.5.2"
3	#import "../lib/code.typ": fixture, isaconst, isai, isalocale, isathm, isatype, listing, thy-badge	3	#import "../lib/code.typ": fixture, isaconst, isai, isalocale, isathm, isatype, listing, thy-badge
	23 unchanged lines hidden		
27	== Encoding the equations for the solver <sec:eq-encoding>	27	== Encoding the equations for the solver <sec:eq-encoding>
28		28	
29	The solver accepts a right-hand side only as a strategy tree, uses one value	29	The solver accepts a right-hand side only as a strategy tree, uses one value
30	type for all unknowns, and allows side effects only to global unknowns. A direct encoding of the equations, along the lines of Apinis et al.	30	type for all unknowns, and allows side effects only to global unknowns. A direct encoding of the equations, as in Apinis et al.
31	and Goblint, needs four things this interface does not offer	31	and Goblint, needs four things this interface does not offer
32	(@tab:eq-adapters). @ch:equations already stated the equations in the adapted	32	(@tab:eq-adapters). @ch:equations already stated the equations in the adapted
33	form, and this section explains why each adaptation is needed. Buffering is the	33	form, and this section explains why each adaptation is needed. Buffering is the
	128 unchanged lines hidden		
162	== Update rules for global unknowns #thy-badge("TD", "Update_rules") <sec:update-rules>	162	== Update rules for global unknowns #thy-badge("TD", "Update_rules") <sec:update-rules>
163		163	
164	A global unknown, such as a seed, receives contributions from several places.	164	A global unknown, such as a seed, receives contributions from several places.
165	The solver merges each new contribution into the value it holds, and this	165	The _update rule_ merges each new contribution into the value the unknown

166	merge, the _update rule_, decides both precision and termination. Tilscher et al. prove five update rules sound against a generic interface	166	holds and decides both precision and termination. Tilscher et al. state the
167	@tilscher26, and Voblint lets the user select any of them.	167	solver's requirements on an update rule as the locale
		168	#isalocale("update_rule") and prove that five rules satisfy it @tilscher26,
		169	so Voblint can offer all five as a user setting
		170	(#isathm("update_rule_update_global_of")). @fig:update-rules compares them.
168		171	
169	The rules differ in what they keep per _origin_, the right-hand side that	172	The rules differ in what they keep per _origin_, the right-hand side that
170	published a contribution (@sec:td), and in whether they widen. Of the two	173	published a contribution (@sec:td), and in whether they widen. Of the two
171	_joining rules_, _join_ joins every contribution into the value, and _join per	174	_joining rules_, _join_ joins every contribution into the value, and _join per
172	origin_ reports the join of each origin's latest contribution. Of the three	175	origin_ joins each origin's latest contribution. Of the three _warrowing rules_,
173	_warrowing rules_, _warrow_ and _warrow per origin_ warrow toward	176	_warrow_ and _warrow per origin_ warrow toward the same join or per origin, and
174	the same join or per origin, and _bounded narrowing_ additionally limits how	177	_bounded narrowing_ also limits how often an origin may switch from widening to
175	often an origin may switch from widening to narrowing. @fig:update-rules runs	178	narrowing. In #isaconst("run_voblint"), the rule governs how a callee's entry
176	all five on one sequence of contributions. In #isaconst("run_voblint"), the rule	179	state and a flow-insensitive program global (@sec:mixed-flow) accumulate their
177	decides how a callee's entry state accumulates across call sites and how a	180	contributions. No rule is more precise on every program (@sec:eval-precision).
178	flow-insensitive program global (@sec:mixed-flow) accumulates its writes. No		
179	rule is more precise on every program, and @sec:eval-precision compares them.		
180		181	
181	#let _rule-steps = update-rule-steps()	182	#let _rule-steps = update-rule-steps()
182		183	
	166 unchanged lines hidden		
349	(#isathm("entry_state_activation_collect_sound_of_terminates")), and the five	350	(#isathm("entry_state_activation_collect_sound_of_terminates")), and the five
350	obligations of @ch:traces hold as @tab:eq-obligations shows. @ch:results turns	351	obligations of @ch:traces hold as @tab:eq-obligations shows. @ch:results turns
351	these states into verdicts.	352	these states into verdicts.

thesis/content/11-instances.typ

d9a59c1f → a1eb4672

414 old lines, 414 new lines

99% similar lines **+4** **-4** **4 changed blocks**

Old	d9a59c1f	New	a1eb4672
1	#import "@preview/fletcher:0.5.8": diagram, edge, node	1	#import "@preview/fletcher:0.5.8": diagram, edge, node
2	#import "../lib/theme.typ": vb	2	#import "../lib/theme.typ": vb
3	#import "../lib/code.typ": fixture, isaconst, isalocale, isathm, isatype, listing, site, thy-badge	3	#import "../lib/code.typ": fixture, isaconst, isalocale, isathm, isatype, listing, site, thy-badge
	9 unchanged lines hidden		
13		13	
14	The preceding chapters stated what a domain, an analysis, and a combination of	14	The preceding chapters stated what a domain, an analysis, and a combination of
15	analyses must prove, and @ch:results showed that every report is sound,	15	analyses must prove, and @ch:results showed that every report is sound,
16	whichever analyses produced it. This chapter introduces and describes the analyses that Voblint currently	16	whichever analyses produced it. This chapter introduces and describes the analyses that Voblint
17	implements. A user selects among the	17	implements. A user selects among the
18	non-relational domains Sign, Interval, Parity, and Congruence, their reduced product	18	non-relational domains Sign, Interval, Parity, and Congruence, their reduced product
19	Int, and a relational order analysis. The chapter compares the four base	19	Int, and a relational order analysis. The chapter compares the four base

	262 unchanged lines hidden		
282	relation. The join of $absval({x <= y, y <= z})$ and $absval({x <= y, x <= z})$ keeps only	282	relation. The join of $absval({x <= y, y <= z})$ and $absval({x <= y, x <= z})$ keeps only
283	their common pair and is $absval({x <= y})$, although both operands imply $x <= z$. The	283	their common pair and is $absval({x <= y})$, although both operands imply $x <= z$. The
284	order analysis is therefore not as precise as a closed relational domain could	284	order analysis is therefore not as precise as a closed relational domain could
285	be. It serves as a small demonstration that the framework supports relational	285	be. It demonstrates that the framework supports relational
286	states.	286	states.
287		287	
288	Pairs enter only at guards, and an assignment to x removes every pair that	288	Pairs enter only at guards, and an assignment to x removes every pair that
	117 unchanged lines hidden		
406		406	
407	The base domains record different properties, Int lets them share these	407	The base domains record different properties, Int lets them share these
408	facts, and the order analysis proves relations that no non-relational	408	facts, and the order analysis proves relations that no non-relational
409	domain can express. In practice, a user need not choose between them. The	409	domain can express. A user does not have to choose between them. The
410	product state and the channel of @ch:cooperation let Int and the order	410	product state and the channel of @ch:cooperation let Int and the order
411	analysis run together. They exchange facts	411	analysis run together. They exchange facts
412	through queries, and every check is decided by the meet of their answers. Selecting both gets the most out of the analyses Voblint currently	412	through queries, and every check is decided by the meet of their answers. Selecting both gets the most out of the analyses Voblint
413	ships, although no theorem guarantees that a combination is more precise than	413	ships, although no theorem guarantees that a combination is more precise than
414	its parts (@sec:coop-mcp).	414	its parts (@sec:coop-mcp).

thesis/content/12-executable.typ

d9a59c1f → a1eb4672

328 old lines, 328 new lines

99% similar lines +2 -2 **2 changed blocks**

Old	d9a59c1f	New	a1eb4672
1	#import "@preview/fletcher:0.5.8": diagram, edge, node	1	#import "@preview/fletcher:0.5.8": diagram, edge, node
2	#import "../lib/math.typ": *	2	#import "../lib/math.typ": *
3	#import "../lib/figures.typ": *	3	#import "../lib/figures.typ": *
	64 unchanged lines hidden		
68	recursive functions translate directly into a functional program, but the	68	recursive functions translate directly into a functional program, but the
69	concretizations and the pointwise states do not. The concretization $conc$ maps an abstract value to a possibly infinite set of integers, and in general	69	concretizations and the pointwise states do not. The concretization $conc$ maps an abstract value to a possibly infinite set of integers, and in general
70	no program can compute such a set. The analyzer does not need to, because	70	no program can compute such a set. The analyzer does not need to, because
71	$conc$ only states what a value means. This is why the domain interface	71	$conc$ only states what a value means. The domain interface therefore
72	separates #isalocale("executable_domain") from the classes and locales	72	separates #isalocale("executable_domain") from the classes and locales
73	below it in @fig:domain-carrier, which add $conc$ and its laws. Code	73	below it in @fig:domain-carrier, which add $conc$ and its laws. Code
74	generation exports the executable part together with each domain's operation	74	generation exports the executable part together with each domain's operation
	89 unchanged lines hidden		
164	are measured on the repository at build time.],	164	are measured on the repository at build time.],
165	) <tab:sizes>	165	) <tab:sizes>
166		166	
167	The proofs say nothing about running time, and the generated code can be very	167	The proofs say nothing about running time, and the generated code can be
168	inefficient. In one case, a sampling profile of 256 chained assignments attributed	168	inefficient. In one case, a sampling profile of 256 chained assignments attributed
169	most of the CPU time to a single function. The generated	169	most of the CPU time to a single function. The generated

170	code of #isaconst("init_publications") recompiled the whole program at every	170	code of #isaconst("init_publications") recompiled the whole program at every
	155 unchanged lines hidden		
326	`known-imprecision/`. #link(repo-blob + "tests/solver-trace")[Golden files] pin	326	`known-imprecision/`. #link(repo-blob + "tests/solver-trace")[Golden files] pin
327	solver traces (@sec:tracing), and property tests check that the parser reads	327	solver traces (@sec:tracing), and property tests check that the parser reads
328	back random syntax trees.	328	back random syntax trees.

thesis/content/13-evaluation.typ

d9a59c1f → a1eb4672

825 old lines, 825 new lines

100% similar lines **+1** **-1** **1 changed blocks**

Old	d9a59c1f	New	a1eb4672
1	#import "../lib/code.typ": *	1	#import "../lib/code.typ": *
2	#import "../lib/stats.typ": stat	2	#import "../lib/stats.typ": stat
3	#import "../lib/math.typ": absval, conc, ivl, parity-name, signval	3	#import "../lib/math.typ": absval, conc, ivl, parity-name, signval
	212 unchanged lines hidden		
216	and #claim-ref("cost-down-k100")). Nearly the whole cost is thus paid before	216	and #claim-ref("cost-down-k100")). Nearly the whole cost is thus paid before
217	any precision is gained.	217	any precision is gained.
218		218	
219	With entry-state contexts, the number of contexts need not even be finite.	219	With entry-state contexts, the number of contexts can even be infinite.
220	Under the unit context policy and bounded call strings, a compiled program has	220	Under the unit context policy and bounded call strings, a compiled program has
221	finitely many unknowns (#isathm("compiled_unit_vars_finite"),	221	finitely many unknowns (#isathm("compiled_unit_vars_finite"),
222	#isathm("compiled_call_strings_finite")). Entry-state contexts under Interval	222	#isathm("compiled_call_strings_finite")). Entry-state contexts under Interval
	600 unchanged lines hidden		
823	growing one program size n under Interval. The fits describe the measured ranges and	823	growing one program size n under Interval. The fits describe the measured ranges and
824	are no complexity bounds.],	824	are no complexity bounds.],
825	) <fig:runtime-scaling>	825	) <fig:runtime-scaling>

thesis/content/15-conclusion.typ

d9a59c1f → a1eb4672

230 old lines, 231 new lines

98% similar lines **+4** **-3** **2 changed blocks**

Old	d9a59c1f	New	a1eb4672
1	#import "../lib/code.typ": isaconst, isalocale, isathm, isatype, oblig, repo-blob, site	1	#import "../lib/code.typ": isaconst, isalocale, isathm, isatype, oblig, repo-blob, site
2	#import "../lib/figures.typ": verdict	2	#import "../lib/figures.typ": verdict
3	#import "../lib/stats.typ": stat	3	#import "../lib/stats.typ": stat
	57 unchanged lines hidden		
61	a similar formalization might do differently.	61	a similar formalization might do differently.
62		62	
63	*Reuse the architecture, but reconsider the model.* Goblint's architecture is	63	*Reuse the architecture, but reconsider the model.* Goblint's architecture is
64	documented in the literature and has been exercised on real programs, so following it closely	64	documented in the literature and has been exercised on real programs. Following
65	avoided designing an analyzer and its soundness proof at the same time. The concrete semantics, however, had to	65	it closely settled the analyzer's design, so the work could concentrate on the
		66	soundness proof. The concrete semantics, however, had to
66	be designed for the proof. The first prototype used whole-program executions	67	be designed for the proof. The first prototype used whole-program executions
67	with calls inlined. To account for one call, the soundness proof built a	68	with calls inlined. To account for one call, the soundness proof built a

68	separate execution starting at the callee's entry and spliced it into the	69	separate execution starting at the callee's entry and spliced it into the
	102 unchanged lines hidden		
171	immediate applicability to Goblint development is limited, although it already	172	immediate applicability to Goblint development is limited, although it already
172	provides a reference for selected components and proof obligations	173	provides a reference for selected components and proof obligations
173	(@sec:outlook-extending). Among the limitations identified in @sec:intro-scope, the gap between VIMP and	174	(@sec:outlook-extending). Among the limitations identified in @sec:intro-scope, the gap between VIMP and
174	C is the most significant. VIMP has no pointers, heap, or threads and	175	C is the largest. VIMP has no pointers, heap, or threads and
175	computes with unbounded integers, so a verdict about a VIMP program does not	176	computes with unbounded integers, so a verdict about a VIMP program does not
176	carry over to a C program with the same text (@sec:vimp-vs-c), and the	177	carry over to a C program with the same text (@sec:vimp-vs-c), and the
177	adequacy of #isaconst("pstep") itself is argued rather than proved. As far as the author is aware, no commonly	178	adequacy of #isaconst("pstep") itself is argued rather than proved. As far as the author is aware, no commonly
	50 unchanged lines hidden		
228	handled the formalization consistently better, and the surrounding	229	handled the formalization consistently better, and the surrounding
229	Isabelle tooling improved as well. Continued progress in both areas may make ambitious formalization projects	230	Isabelle tooling improved as well. Continued progress in both areas may make ambitious formalization projects
230	and large-scale refactorings increasingly practical.	231	and large-scale refactorings increasingly practical.

thesis/content/abstract.typ

d9a59c1f → a1eb4672

64 old lines, 64 new lines

98% similar lines **+1** **-1** **1 changed blocks**

Old	d9a59c1f	New	a1eb4672
1	#import "../lib/code.typ": site	1	#import "../lib/code.typ": site
2	#import "../lib/stats.typ": stat	2	#import "../lib/stats.typ": stat
3	#import "../lib/figures.typ": verdict	3	#import "../lib/figures.typ": verdict
	22 unchanged lines hidden		
26	the solver is not proved. A check reported #verdict("DEAD") is also proved	26	the solver is not proved. A check reported #verdict("DEAD") is also proved
27	unreachable. At a point without an arithmetic warning or error, no divisor is	27	unreachable. At a point without an arithmetic warning or error, no divisor is
28	zero, and a divisor reported as an error is zero wherever its point is reached.	28	zero, and a divisor reported as an error is zero wherever its point is reached.
29	To the author's knowledge, Voblint is the first mechanized analyzer whose soundness proof	29	To the author's knowledge, no prior mechanized analyzer has a soundness proof that
30	connects a source semantics, through a context-indexed collecting semantics, to	30	connects a source semantics, through a context-indexed collecting semantics, to
31	a side-effecting constraint system and its verified solver.	31	a side-effecting constraint system and its verified solver.
32		32	
	29 unchanged lines hidden		
62	generator, and the OCaml and WebAssembly toolchains are trusted and tested	62	generator, and the OCaml and WebAssembly toolchains are trusted and tested
63	conventionally using unit and regression tests. Because the theorem concerns VIMP, VIMP's differences from	63	conventionally using unit and regression tests. Because the theorem concerns VIMP, VIMP's differences from
64	C11 @iso-c11 are documented.	64	C11 @iso-c11 are documented.

thesis/content/ai-use.typ

d9a59c1f → a1eb4672

131 old lines, 128 new lines

85% similar lines **+18** **-21** **4 changed blocks**

Old	d9a59c1f	New	a1eb4672
1	#import "../lib/stats.typ": _grouped as _n	1	#import "../lib/stats.typ": _grouped as _n
2		2	

Old	d9a59c1f	New	a1eb4672
3	// Aggregates only, written by tools/ai_use_stats.py from local session logs.	3	// Aggregates only, written by tools/ai_use_stats.py from local session logs.
	52 unchanged lines hidden		
56	treated as a proposal until Isabelle, a test or the author's review confirmed it. This	56	treated as a proposal until Isabelle, a test or the author's review confirmed it. This
57	caution was necessary, since the assistants' output was not reliable on its own. Agents proposed incorrect	57	caution was necessary, since the assistants' output was not reliable on its own. Agents proposed incorrect
58	lemmas and, in one case, introduced a premise into the end-to-end soundness theorem that	58	lemmas and, in one case, introduced a premise into the end-to-end soundness theorem that
59	no program satisfied, making the theorem vacuous until the solver run was	59	no program satisfied, so the theorem was vacuous until the solver run was
60	redesigned. Audits of theorem statements and full batch builds exposed these	60	redesigned. Audits of theorem statements and full batch builds exposed these
61	errors. The non-vacuity instances of @sec:nonvacuity now demonstrate that the	61	errors. The non-vacuity instances of @sec:nonvacuity now demonstrate that the
62	premises of the end-to-end soundness theorem and its companions can be satisfied.	62	premises of the end-to-end soundness theorem and its companions can be satisfied.
63		63	
64	Agents also wrote much of the tooling around the formalization, such as the	64	Agents also wrote much of the tooling around the formalization, such as the
65	command-line tool, the project site, and the regression corpus with its ports of	65	command-line tool, the project site, and the regression corpus with its ports of
66	Goblint regression tests. They also wrote the checks that keep the theories, the	66	Goblint regression tests. They also wrote the checks that keep the theories, the
67	generated material, the analyzer, the project site, and this thesis consistent	67	generated material, the analyzer, the project site, and this thesis consistent
68	(@sec:voblint-repo). These checks proved especially useful because they replaced	68	(@sec:voblint-repo). These checks proved especially useful because they replaced
69	repeated manual inspection with conditions that every change had to satisfy, and	69	repeated manual inspection with conditions that every change had to satisfy, and
70	they exposed errors that manual reading had missed.	70	they exposed errors that manual reading had missed.
71		71	
72	The table below quantifies the recorded part of this assistance, following the	72	@fig:ai-sessions quantifies the recorded part of this assistance using
73	session-log analysis of Bryant et al. @bryant26munkres[Section 5]. The counts	73	session-log counts, following Bryant et al. @bryant26munkres[Section 5]. The counts
74	measure interaction with the tools, not authorship or the proportion of the work	74	measure interaction with the tools, not authorship or the proportion of the work
75	attributable to them.	75	attributable to them.
76		76	
77	#let _cc = _ai.claude_code	77	#let _cc = _ai.claude_code
78	#let _cx = _ai.codex	78	#let _cx = _ai.codex
79	#let _row(label, key, codex: true) = (	79	#let _row(label, key, codex: true) = (
80	label,	80	label,
81	..(_cc.top_level, _cc.subagents).map(s => _n(s.at(key))),	81	..(_cc.top_level, _cc.subagents).map(s => _n(s.at(key))),
82	..(_cx.top_level, _cx.subagents).map(s => if codex { _n(s.at(key)) } else [--]),	82	..(_cx.top_level, _cx.subagents).map(s => if codex { _n(s.at(key)) } else [--]),
83	)	83	)
84	#v(0.4em)	84	#figure(
85	#block(breakable: false, width: 100%)[	85	table(
86	#align(center, table(		
87	columns: 5,	86	columns: 5,
88	align: (left, right, right, right, right),	87	align: (left, right, right, right, right),
89	stroke: none,	88	stroke: none,
	10 unchanged lines hidden		
100	.._row([#h(1em) Isabelle server (I/Q, PIDE)], "isabelle_mcp"),	99	.._row([#h(1em) Isabelle server (I/Q, PIDE)], "isabelle_mcp"),
101	.._row([#h(1em) file edits and writes], "edits"),	100	.._row([#h(1em) file edits and writes], "edits"),
102	table.hline(),	101	table.hline(),
103	))	102	),
104	#v(0.4em)	103	kind: table,

105	#set text(size: 0.9em)	104	caption: [Recorded automated-assistant activity during development, counted on #_ai.snapshot from the
106	Recorded automated-assistant activity during development, counted on #_ai.snapshot from the	105	local session logs (#_cc.top_level.first_day to #_cc.top_level.last_day for
107	local session logs (#_cc.top_level.first_day to #_cc.top_level.last_day for	106	Claude Code, #_cx.top_level.first_day to #_cx.top_level.last_day for Codex).
108	Claude Code, #_cx.top_level.first_day to #_cx.top_level.last_day for Codex).	107	No local logs are available for earlier Claude Code sessions, ChatGPT in
109	No local logs are available for earlier Claude Code sessions, ChatGPT in	108	the browser, or Cursor, so these are not counted. The author sent
110	the browser, or Cursor, so these are not counted. The author sent	109	#_n(_cc.top_level.human_messages) messages to Claude Code and
111	#_n(_cc.top_level.human_messages) messages to Claude Code and	110	#_n(_cx.top_level.human_messages) to Codex. Shell edits count as shell calls.
112	#_n(_cx.top_level.human_messages) to Codex. Shell edits count as shell calls.	111	Interactions through the Isabelle server, including theory edits, count as
113	Interactions through the Isabelle server, including theory edits, count as	112	server calls. Co-author lines in commit messages were not kept consistently
114	server calls. Co-author lines in commit messages were not kept consistently	113	and are not used as a measure.],
115	and are not used as a measure.	114	) <fig:ai-sessions>
116	]		
117	#v(0.8em)		
118		115	
119	Isabelle/HOL accepts every theorem in the development. Proofs by evaluation,	116	Isabelle/HOL accepts every theorem in the development. Proofs by evaluation,
120	used for some facts about fixed programs, additionally rely on Isabelle's code	117	used for some facts about fixed programs, additionally rely on Isabelle's code
121	generator (@fig:intro-trust). This establishes the stated propositions	118	generator (@fig:intro-trust). This establishes the stated propositions
122	relative to their definitions and assumptions, but does not establish that those	119	relative to their definitions and assumptions, but does not establish that those
123	definitions faithfully capture the intended language or analysis. Definitions	120	definitions faithfully capture the intended language or analysis. Definitions
124	and theorem statements were therefore also checked against example programs, the	121	and theorem statements were therefore also checked against example programs, the
125	regression corpus, and the documented differences between VIMP and C11, much as a human	122	regression corpus, and the documented differences between VIMP and C11, much as a human
126	audit checks the statements of AI-generated formal proofs @hales26lean[Sec. "Is Lean reliable?"].	123	audit checks the statements of AI-generated formal proofs @hales26lean[Sec. "Is Lean reliable?"].
127	Related-work claims were checked against the cited sources, often with agent	124	Related-work claims were checked against the cited sources, often with agent
128	assistance and subsequent author review.	125	assistance and subsequent author review.
129		126	
130	Responsibility for the final claims, framing, and treatment of related work lies	127	Responsibility for the final claims, framing, and treatment of related work lies
131	with the author alone.	128	with the author alone.

thesis/content/appendix-tooling.typ

d9a59c1f → a1eb4672

268 old lines, 266 new lines

97% similar lines +7 -9 **6 changed blocks**

Old	d9a59c1f	New	a1eb4672
1	#import "@preview/fletcher:0.5.8": diagram, edge, node	1	#import "@preview/fletcher:0.5.8": diagram, edge, node
2	#import "../lib/code.typ": isa, isacmd, isaconst, isalocale, isathm, repo-blob, site	2	#import "../lib/code.typ": isa, isacmd, isaconst, isalocale, isathm, repo-blob, site
3	#import "../lib/theme.typ": vb	3	#import "../lib/theme.typ": vb
4	#import "../lib/stats.typ": stat	4	#import "../lib/stats.typ": stat
5		5	
6	= Repository and Tooling <sec:voblint-repo>	6	= Repository and Tooling <sec:voblint-repo>
7		7	
8	The theories, the exported analyzer, the project site, and this thesis quote	8	The theories, the exported analyzer, the project site, and this thesis quote
9	one another. The analyzer is exported from the theories, the thesis cites	9	one another. The analyzer is exported from the theories, the thesis cites

Old	d9a59c1f	New	a1eb4672
10	their names and statements, and both the site and the thesis quote the	10	their names and statements, and both the site and the thesis quote the
11	analyzer's output. Every change to the theories can therefore make another	11	analyzer's output. Every change to the theories can therefore make another
12	part stale, and keeping them consistent is engineering work outside the proof.	12	part stale, and keeping them consistent is engineering work outside the proof.
13	Isabelle offers comparatively little project-level tooling for this beyond its	13	Isabelle offers comparatively little project-level tooling for this beyond its
14	distribution, a gap that users have noted as well @paulson26lean	14	distribution. Users have pointed this out as well @paulson26lean
15	(#link("https://github.com/lawrencecpaulson/lawrencecpaulson.github.io/issues/62#issuecomment-4333878708")[reader comment]). The need grew with AI assistance. Agents wrote many proofs	15	(#link("https://github.com/lawrencecpaulson/lawrencecpaulson.github.io/issues/62#issuecomment-4333878708")[reader comment]). The need grew with AI assistance. Agents wrote many proofs
16	and much of the surrounding code (#link(<ai-use>)[statement on the use of	16	and much of the surrounding code (#link(<ai-use>)[statement on the use of
17	generative AI]), and a reviewer cannot read every line they produce. A change	17	generative AI]), and a reviewer cannot read every line they produce. A change
	31 unchanged lines hidden		
49	#let _cf-ready = _cf.values().filter(p => p.available).len()	49	#let _cf-ready = _cf.values().filter(p => p.available).len()
50	#let _pr(n) = "https://github.com/conda-forge/staged-recipes/pull/" + str(n)	50	#let _pr(n) = "https://github.com/conda-forge/staged-recipes/pull/" + str(n)
51		51	
52	The repository currently relies on several package managers.	52	The repository relies on several package managers.
53	#link("https://pixi.sh")[Pixi] installs Python and the conda-forge packages	53	#link("https://pixi.sh")[Pixi] installs Python and the conda-forge packages
54	and runs every task, opam installs the OCaml toolchain for the exported	54	and runs every task, opam installs the OCaml toolchain for the exported
55	analyzer and its browser build, and Isabelle is installed separately. The goal	55	analyzer and its browser build, and Isabelle is installed separately. The goal
56	is a single Pixi file from which one `pixi install` sets up everything. For the	56	is a single Pixi file from which one `pixi install` sets up everything. For the
57	OCaml side, this requires every package the analyzer builds with on	57	OCaml side, this requires every package the analyzer builds with on
58	conda-forge for all supported platforms. Until recently, however, conda-forge	58	conda-forge for all supported platforms. Until recently, however, conda-forge
59	carried the OCaml compiler and only a handful of tools, such as Dune and	59	carried the OCaml compiler and only a few tools, such as Dune and
60	Menhir, and almost none of the libraries below them.	60	Menhir, and almost none of the libraries below them.
61		61	
62	Alongside this thesis, considerable effort went into changing this, together	62	Alongside this thesis, considerable effort went into changing this, together
63	with other conda-forge contributors. A #link(_pr(34872))[recipe for ten base	63	with other conda-forge contributors. A #link(_pr(34872))[recipe for ten base
64	libraries], from `csexp` to `yojson`, has been merged, and Apple Silicon builds	64	libraries], from `csexp` to `yojson`, has been merged, and Apple Silicon builds
65	have been enabled for each of them. Two fixes to the Dune package have been	65	have been enabled for each of them. Two fixes to the Dune package have been
66	merged, and recipes for `ppxlib` and `sedlex`, as well as an update to Zarith,	66	merged, and recipes for `ppxlib` and `sedlex`, as well as an update to Zarith,
67	are under review. `isar-tools` itself #link(_pr(34960))[installs from conda-forge]. Of	67	are under review. `isar-tools` itself #link(_pr(34960))[installs from conda-forge]. Of
68	the #_cf.len() packages in the	68	the #_cf.len() packages in the
69	#link(repo-blob + "manifests/conda-forge-ocaml.toml")[toolchain manifest],	69	#link(repo-blob + "manifests/conda-forge-ocaml.toml")[toolchain manifest],
70	#_cf-ready are now available on conda-forge, and the others, among them	70	#_cf-ready are now available on conda-forge, and the others, among them
71	`js_of_ocaml` and `ocamlformat`, still come from opam. As far as the author is	71	`js_of_ocaml` and `ocamlformat`, still come from opam.
72	aware, this is the first larger effort to bring OCaml libraries to		
73	conda-forge.		
74		72	
75	== Derived files and checks	73	== Derived files and checks
76		74	
	99 unchanged lines hidden		
176	kind: image,	174	kind: image,
177	placement: auto,	175	placement: auto,

178	caption: [The sources of the repository (blue) and the files derived from	176	caption: [The sources of the repository (blue) and the files derived from
179	them; each box links to its file. Each derived file is generated during the build or committed with a	177	them. Each box links to its file. Each derived file is generated during the build or committed with a
180	check that regenerates it; the link map is also checked against the	178	check that regenerates it. The link map is also checked against the
181	published site.],	179	published site.],
182	) <fig:pipeline>	180	) <fig:pipeline>
183		181	
	22 unchanged lines hidden		
206	written for this thesis and published separately, addresses them.	204	written for this thesis and published separately, addresses them.
207	The command-line tool reads theory and `ROOT` files the way Isabelle's build	205	The command-line tool reads theory and `ROOT` files the way Isabelle's build
208	finds them and never runs Isabelle itself, so it answers in seconds, fits a	206	finds them and never runs Isabelle itself, so it answers in seconds, fits a
209	pre-commit hook, and also works on theories that do not yet build. Related	207	pre-commit hook, and works on theories that do not yet build. Related
210	tools cover parts of this ground. The	208	tools cover parts of this ground. The
211	#link("https://github.com/isabelle-prover/isabelle-linter")[Isabelle Linter]	209	#link("https://github.com/isabelle-prover/isabelle-linter")[Isabelle Linter]
212	checks proof style inside Isabelle, and	210	checks proof style inside Isabelle, and
	53 unchanged lines hidden		
266	suspicious patterns and structural problems, and they may report false	264	suspicious patterns and structural problems, and they may report false
267	positives or miss problems. Whether a theory is formally correct is still	265	positives or miss problems. Whether a theory is formally correct is still
268	decided by the Isabelle build alone.	266	decided by the Isabelle build alone.

thesis/content/reading-guide.typ

d9a59c1f → a1eb4672

91 old lines, 91 new lines

99% similar lines **+1** **-1** **1 changed blocks**

Old	d9a59c1f	New	a1eb4672
1	#import "../lib/math.typ": *	1	#import "../lib/math.typ": *
2	#import "../lib/theme.typ": vb	2	#import "../lib/theme.typ": vb
3	#import "../lib/code.typ": (	3	#import "../lib/code.typ": (
	14 unchanged lines hidden		
18	profiles, and the #link(site("playground.html"))[playground], which runs the analyzer in	18	profiles, and the #link(site("playground.html"))[playground], which runs the analyzer in
19	the browser on any VIMP program.	19	the browser on any VIMP program.
20		20	
21	The table below lists how the text marks each kind of object. Most markers are also links.	21	Each kind of object the text refers to has its own marker, and most markers are also links.
22	An Isabelle name opens its definition, a program opens the playground, and a fixture opens	22	An Isabelle name opens its definition, a program opens the playground, and a fixture opens
23	its file in the repository.	23	its file in the repository.
24		24	
	64 unchanged lines hidden		
89	table.hline(),	89	table.hline(),
90	)	90	)
91	}	91	}

thesis/lib/tum.typ

d9a59c1f → a1eb4672

457 old lines, 464 new lines

97% similar lines **+18** **-11** **10 changed blocks**

Old	d9a59c1f	New	a1eb4672
1	#import "@preview/rich-counters:0.2.2": rich-counter	1	#import "@preview/rich-counters:0.2.2": rich-counter
2	#import "@preview/hydra:0.6.3": hydra	2	#import "@preview/hydra:0.6.3": hydra
3	#import "theme.typ": vb	3	#import "theme.typ": vb
	122 unchanged lines hidden		
126	logo: "../assets/tumlogo.svg",	126	logo: "../assets/tumlogo.svg",
127	body,	127	body,
128	) = {	128	) = {
129	set document(title: title, author: author)	129	// The commit a build came from, in the PDF metadata only, so feedback on an
		130	// old copy can be traced back to its sources.
		131	set document(
		132	title: title,
		133	author: author,
		134	keywords: if "revision" in sys.inputs { ("revision " + sys.inputs.revision,) } else { () },
		135	)
130		136	
131	set page(	137	set page(
132	paper: "a4",	138	paper: "a4",
	98 unchanged lines hidden		
231	// In a *reference* it runs at the reference site, which silently renders a	237	// In a *reference* it runs at the reference site, which silently renders a
232	// chapter-3 figure as "Figure 1.1" when cited from chapter 1 -- so	238	// chapter-3 figure as "Figure 1.1" when cited from chapter 1 -- so
233	// references are resolved separately, in the `show ref` rule below.	239	// references are resolved separately, in the `show ref` rule below.
234	let chapter-number(loc) = {	240	// Front-matter chapters are unnumbered, so their figures carry no prefix.
		241	let figure-number(loc, n) = {
235	let c = counter(heading).at(loc).at(0, default: 0)	242	let c = counter(heading).at(loc).at(0, default: 0)
236	let chapters = query(heading.where(level: 1).before(loc))	243	let chapters = query(heading.where(level: 1).before(loc))
237	let h = chapters.last(default: none)	244	let h = chapters.last(default: none)
238	if h != none and type(h.numbering) == str and h.numbering.starts-with("A") {	245	if h == none or h.numbering == none {
239	numbering("A", c)	246	str(n)
		247	} else if type(h.numbering) == str and h.numbering.starts-with("A") {
		248	numbering("A", c) + "." + str(n)
240	} else {	249	} else {
241	str(c)	250	str(c) + "." + str(n)
242	}	251	}
243	}	252	}
244	set figure(numbering: n => context [#chapter-number(here()).#n])	253	set figure(numbering: n => context figure-number(here(), n))
245		254	
246	show ref: it => {	255	show ref: it => {
247	let el = it.element	256	let el = it.element
	16 unchanged lines hidden		
264	}	273	}
265	// Resolve both halves of the number where the figure is, not where the	274	// Resolve both halves of the number where the figure is, not where the
266	// sentence citing it happens to sit.	275	// sentence citing it happens to sit.

```
267     let chapter = chapter-number(loc)
268     // A subpar part is the only unlisted figure. Its own counter may be of
269     // another kind (a listing inside an image figure), so it takes the number
270     // of the enclosing figure and its position among the parts.
271     if el.outlined == false {
272       let parent = query(figure.where(kind: image, outlined: true).before(loc)).last()
273       let n = counter(figure.where(kind: image)).at(parent.location()).first()
274       let sub = counter("__subpar:sub-figure-counter").at(loc).first() + 1
275       return link(loc, [#el.supplement #chapter.#n#numbering("a", sub)])
276     }
277     let n = counter(figure.where(kind: el.kind)).at(loc).at(0, default: 0)
278     link(loc, [#el.supplement #chapter.#n])
279   }
280   // Algorithms carry their caption above the body, as algorithm2e does.
281   show figure.where(kind: "algorithm"): set figure.caption(position: top)
```

```
276     // A subpar part is the only unlisted figure. Its own counter may be of
277     // another kind (a listing inside an image figure), so it takes the number
278     // of the enclosing figure and its position among the parts.
279     if el.outlined == false {
280       let parent = query(figure.where(kind: image, outlined: true).before(loc)).last()
281       let n = counter(figure.where(kind: image)).at(parent.location()).first()
282       let sub = counter("__subpar:sub-figure-counter").at(loc).first() + 1
283       return link(loc, [#el.supplement #figure-number(loc, n)#numbering("a", sub)])
284     }
285     let n = counter(figure.where(kind: el.kind)).at(loc).at(0, default: 0)
286     link(loc, [#el.supplement #figure-number(loc, n)])
287   }
288   // Algorithms carry their caption above the body, as algorithm2e does.
289   show figure.where(kind: "algorithm"): set figure.caption(position: top)
```

22 unchanged lines hidden

```
304     let el = it.element
305     if el.func() == figure and el.numbering != none {
306       let loc = el.location()
307       let chapter = chapter-number(loc)
308       let n = counter(figure.where(kind: el.kind)).at(loc).at(0, default: 0)
309       // The list is rendered after the chapters; resolve at the figure itself.
310       return link(loc, it.indented([#el.supplement #chapter.#n], it.inner()))
311     }
312     if el.func() != heading {
313       return it
```

```
312     let el = it.element
313     if el.func() == figure and el.numbering != none {
314       let loc = el.location()
315       let n = counter(figure.where(kind: el.kind)).at(loc).at(0, default: 0)
316       // The list is rendered after the chapters; resolve at the figure itself.
317       return link(loc, it.indented([#el.supplement #figure-number(loc, n)], it.inner()))
318     }
319     if el.func() != heading {
320       return it
```

141 unchanged lines hidden

```
455   set page(numbering: "1")
456   counter(page).update(1)
457 }
```

```
462   set page(numbering: "1")
463   counter(page).update(1)
464 }
```